面向对象分析与设计
Object Oriented Analysis and Design

—— 软件体系结构
Software Architecture

邱明 博士

厦门大学信息学院

mingqiu@xmu.edu.cn

2026 年秋季学期

课程提纲
Course Outline

1

基础与定位

体系结构定义
为什么需要架构
视图与利益相关者

第3—10页

2

架构风格

组件、连接件与约束
分层与六边形架构
事件驱动与服务化

第11—19页

3

质量属性与架构权衡

质量属性
质量场景
架构战术与权衡

第20—22页

4

描述、评价与治理

多视图与C4
ADR与接口契约
体系结构评价

第23—26页

课程主线：架构定位 → 风格选择 → 质量权衡 → 描述评价与持续治理

1 基础与定位

什么是软件体系结构

为什么需要软件体系结构

Why Architecture Matters

控制复杂度

通过组件边界、职责和依赖规则控制理解成本。

保障质量属性

性能、可用性、安全性、可修改性主要由架构决定。

促进沟通

为业务、开发、测试和运维提供共同语言。

支持演进

让变化被限制在边界内，避免全局修改。

体系结构不是一张图，而是一组影响全局质量与演化的关键决策。

为什么需要体系结构：控制复杂性

Why Architecture: Managing Complexity

没有体系结构时

- 依赖关系隐藏在代码中
- 修改一个需求牵动多个模块
- 新人需要理解整个系统
- 技术选择散落在各处
- 运行时与部署关系难以解释

体系结构提供的能力

- 明确组件边界和职责
- 规定依赖方向和访问规则
- 把变化限制在局部
- 让理解系统只需关注相关视图
- 用共同模型讨论复杂系统

体系结构通过“分而治之 + 约束依赖”控制认知负荷和变更成本。

为什么需要体系结构：保障质量属性

Why Architecture: Quality Attributes

性能、可用性、安全和可修改性不是编码完成后再补上的功能

性能

响应时间、吞吐量和资源效率。

缓存、并发、异步、分片

可用性

故障恢复、冗余和降级能力。

复制、健康检查、熔断

安全性

身份、权限、数据和审计。

边界防护、最小权限、加密

可修改性

变化影响范围和修改成本。

低耦合、信息隐藏、稳定接口

可测试性

系统能否被隔离和验证。

依赖注入、端口适配器

可部署性

发布、回滚和环境一致性。

容器化、自动化、蓝绿发布

质量属性通常是冲突的；架构设计就是明确优先级、选择战术并管理权衡。

为什么需要体系结构：支持演进

Why Architecture: Enabling Evolution

降低变更成本

变化被限制在模块边界内，减少全局修改。

支持独立开发

清晰边界让多个团队可以并行工作。

保护关键资产

领域规则和数据所有权不被技术细节污染。

允许渐进替换

通过稳定接口替换数据库、框架或外部系统。

控制技术债

架构适应度函数持续检查依赖和约束。

保留架构跑道

为未来需求预留演进空间，而不是提前过度设计。

体系结构不是预测所有未来，而是让重要变化可以在边界内发生。

软件体系结构的定义
Defining Software Architecture

系统的基本组织，体现为组件、组件之间的关系、环境约束，以及指导设计和演化的原则。

组件 Components

承担职责、封装状态、提供接口。

连接件 Connectors

调用、消息、事件、数据流、共享数据。

配置 Configuration

组件如何连接、分层、组合和约束。

体系结构与详细设计
Architecture vs Detailed Design

体系结构

- 系统级组件和边界
- 跨模块依赖和通信
- 质量属性与关键权衡
- 技术选型和部署约束
- 长期演化方向

详细设计

- 类、接口和数据结构
- 算法和局部控制流程
- 事务与异常细节
- 单元级职责
- 可测试性和局部性能

边界不是绝对的：架构决策约束详细设计，详细设计反馈架构可行性。

体系结构视图与利益相关者

Views and Stakeholders

逻辑视图	开发视图	运行视图	物理视图	场景视图
功能、领域模型、模块职责	代码组织、组件、依赖、构建	进程、线程、并发、消息与部署	服务器、网络、容器、存储	关键用例贯穿并验证其他视图
业务与设计人员	开发与测试人员	运维与SRE	基础设施团队	全部利益相关者

2 架构风格

组件、连接件与约束的组合

架构风格：组件、连接件与约束

Architecture Styles

组件

系统的计算和状态单元。
例：服务、模块、过滤器、数据库。

连接件

组件之间的交互机制。
例：调用、事件、管道、共享数据。

约束

对连接、依赖、层次和部署的限制。
例：只能逐层访问、依赖指向内部。

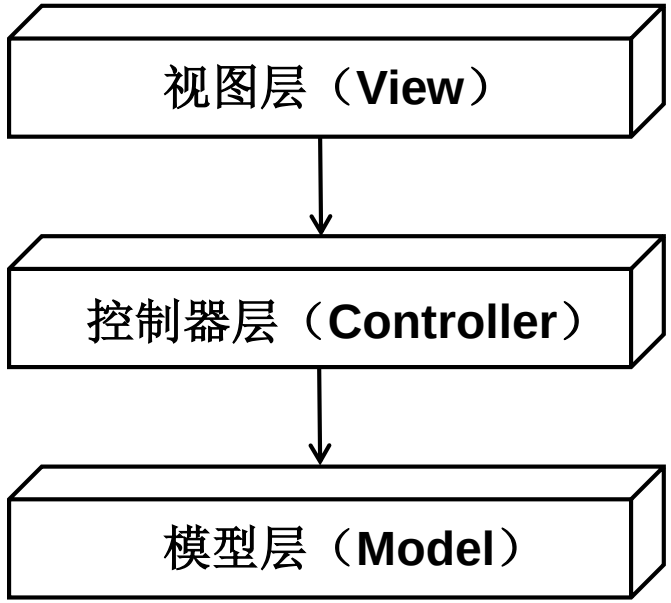
权衡

每个风格优化某些质量属性，同时牺牲其他质量属性。

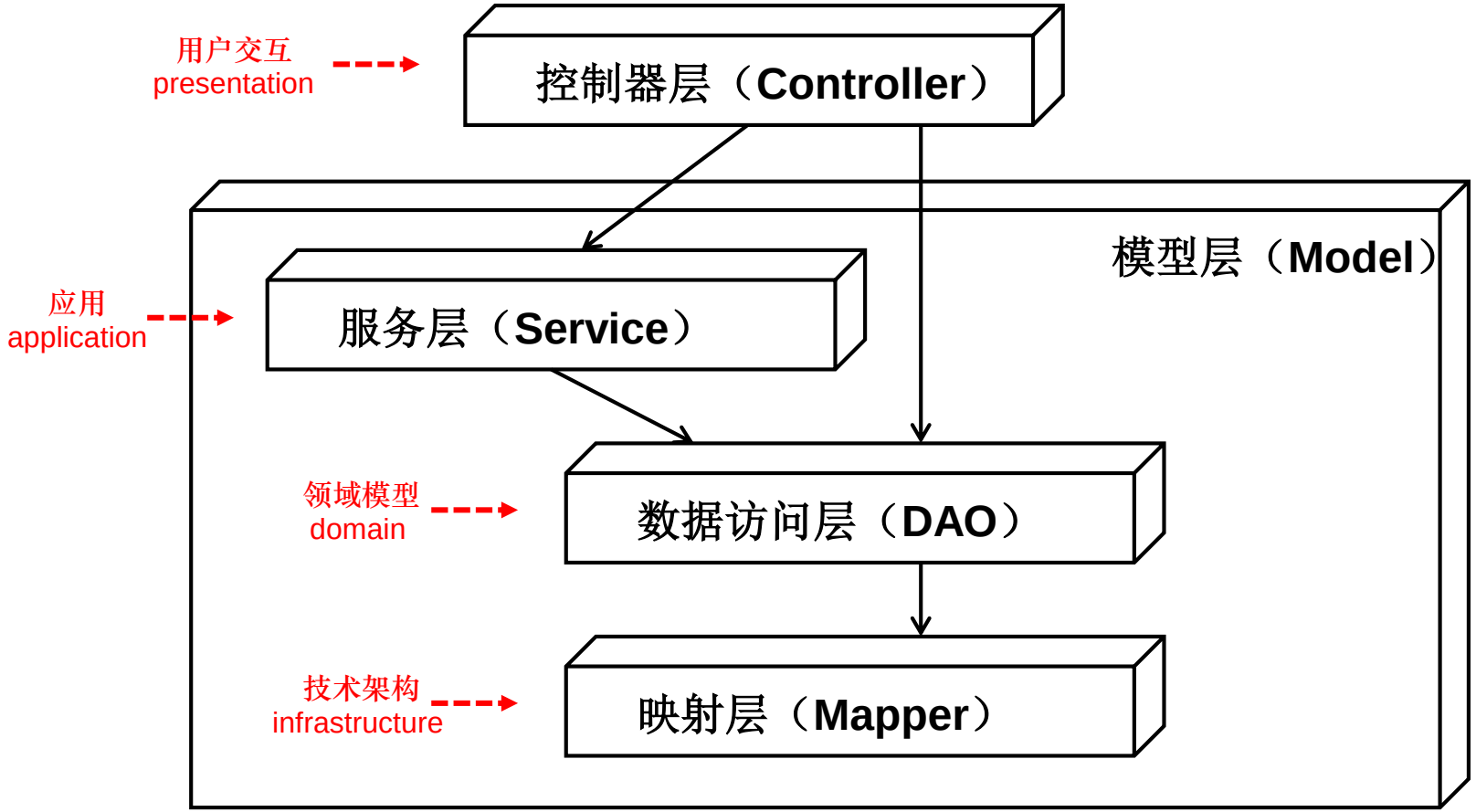
架构风格不是模板，而是约束语言；选择风格就是在选择一组权衡。

分层架构：基本思想
Layered Architecture

- 大粒度的系统划分方式，每层具备清晰的职责，具备内聚性
- 层内耦合大，层间耦合小
  - 变更控制在一层之内
  - 不同的逻辑可以分开

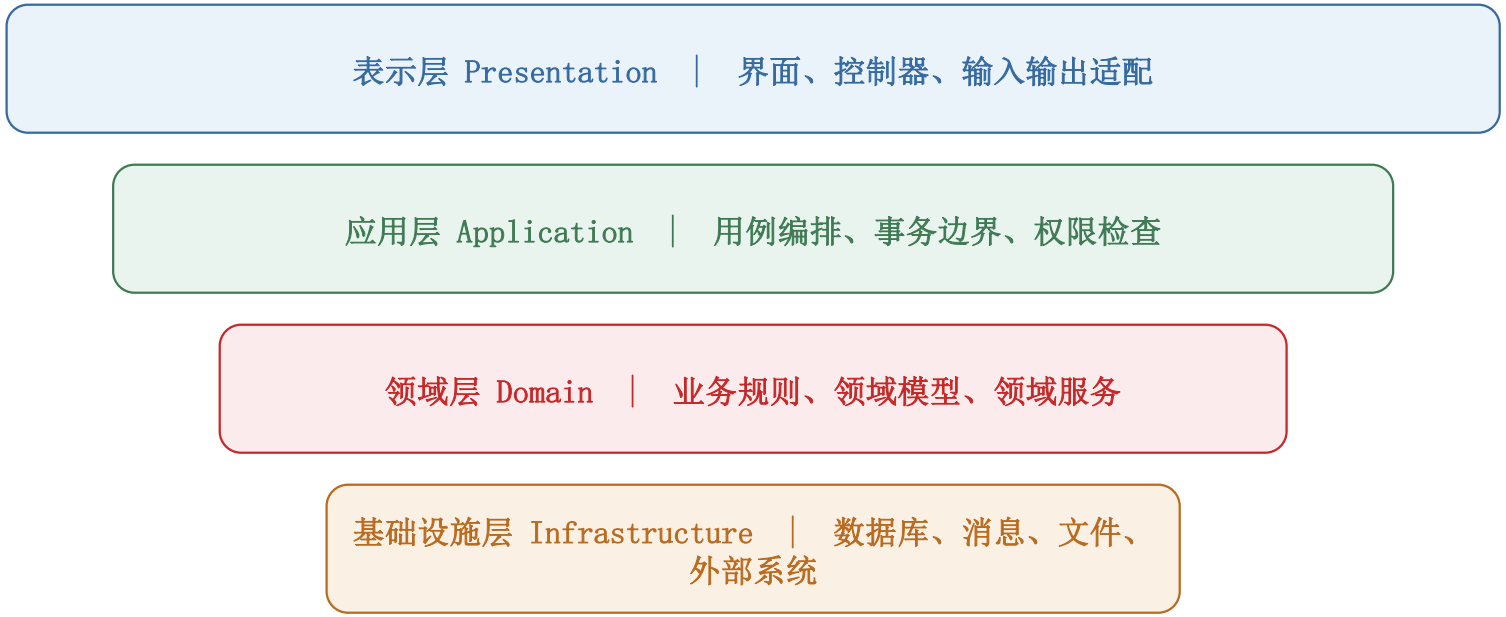


分层架构：典型调用结构
Layered Architecture



分层架构：层次职责与依赖规则

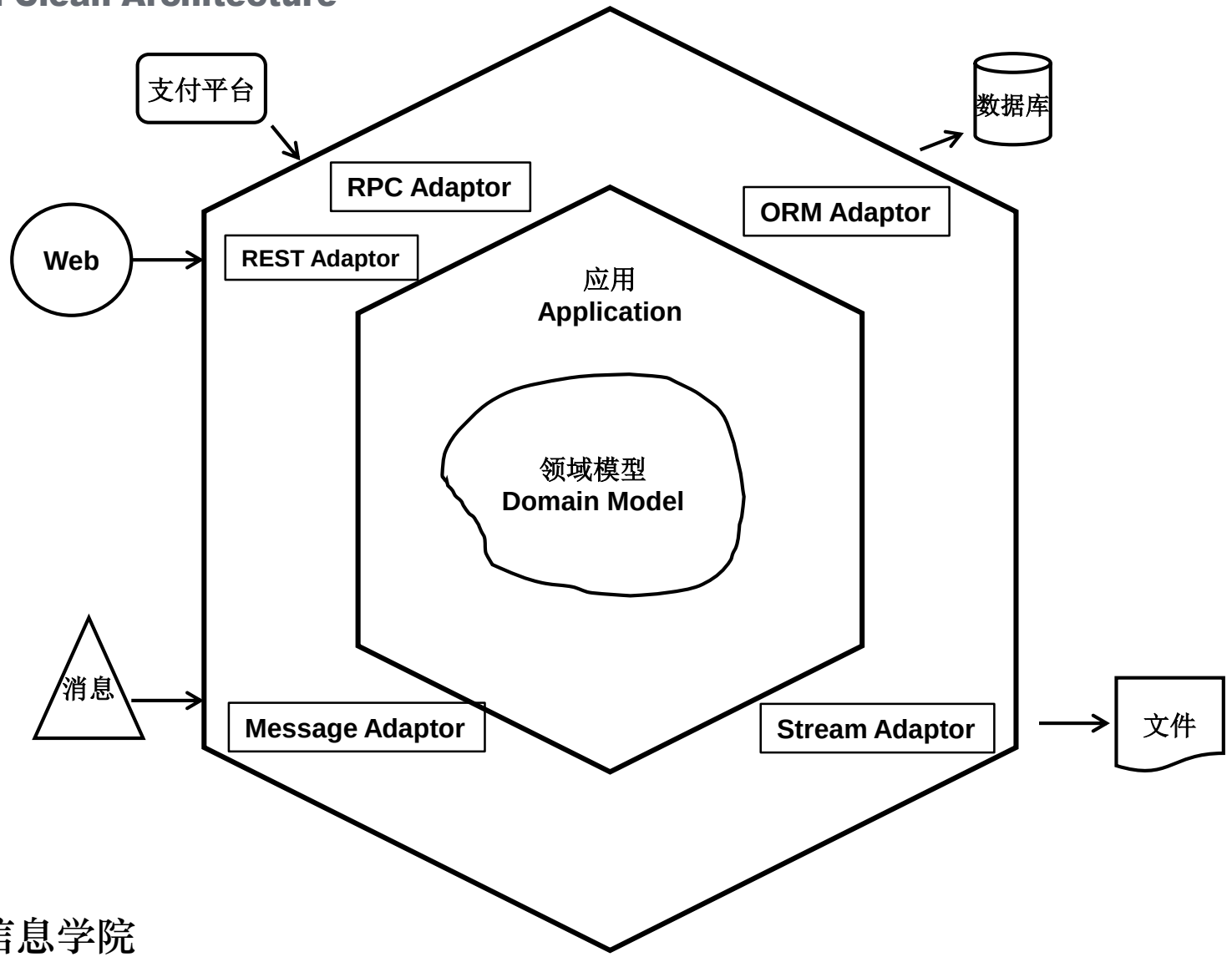
Layered Architecture



规则：上层依赖下层；领域层不依赖具体数据库和框架。

六边形架构与整洁架构

Hexagonal and Clean Architecture



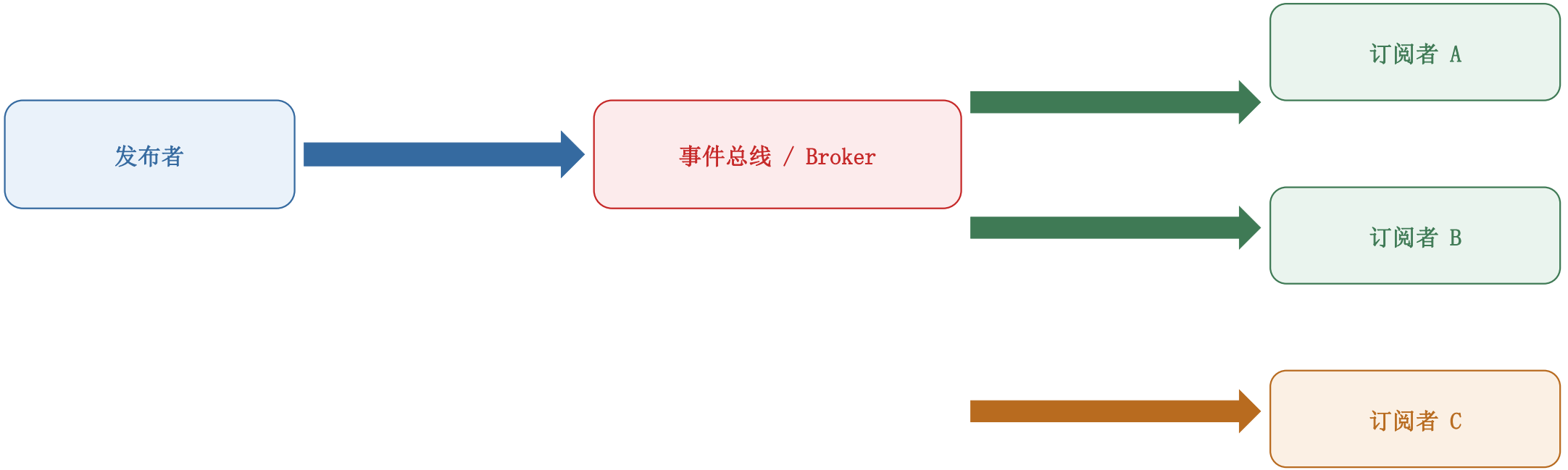
管道—过滤器风格
Pipe-and-Filter

数据流经过独立过滤器，每个过滤器只负责一种转换。



适合：ETL、编译器、数据管道、流处理。
风险：端到端延迟、调试和错误定位成本上升。

事件驱动架构
Event-Driven Architecture



事件是已经发生的业务事实：OrderCreated、PaymentCompleted、ShipmentCreated。

优势：解耦、异步、可扩展；代价：最终一致性、幂等、重放和可观测性。

组件、服务与微服务

Components, Services and Microservices

组件

逻辑边界
可替换单元
通过接口协作
不一定是独立进程

服务

独立部署
明确接口
拥有业务能力
可独立扩展

微服务

围绕业务能力
独立数据所有权
团队自治
异步协作

先模块化，再服务化；不要跳过边界设计直接拆微服务。

3 质量属性与架构权衡

从质量属性到架构战术与权衡

软件质量属性
Quality Attributes

可用性

故障恢复、冗余、健康检查、降级。

性能

延迟、吞吐、资源效率、缓存与并发。

安全性

认证、授权、加密、审计、最小权限。

可修改性

内聚、低耦合、信息隐藏、稳定接口。

可测试性

可隔离、依赖可替换、可观测。

可部署性

自动发布、回滚、环境一致、配置治理。

质量属性通常是相互冲突的；架构设计就是明确优先级并管理权衡。

架构战术与权衡
Tactics and Tradeoffs

典型架构战术

- 缓存、复制、分片
- 超时、重试、熔断、降级
- 认证、授权、加密、审计
- 接口隔离、适配器、事件
- 自动部署、蓝绿发布、回滚

战术会带来代价

性能战术可能增加一致性成本
可用性战术可能增加复杂度
安全战术可能增加延迟
解耦战术可能增加运维难度
演进战术可能增加短期交付成本

没有免费的架构质量：每个战术都要评估收益、代价和适用边界。

4 描述、评价与治理

把架构决策变得可沟通、可验证、可演进

架构描述：多视图表达不同关切

Describing Architecture with Multiple Views

逻辑视图

职责、领域模型
模块与依赖关系

运行视图

进程、线程、并发
消息与调用时序

开发视图

代码组织、组件
构建与团队边界

部署视图

节点、容器、网络
环境与部署拓扑

视图不是为了画全，而是按受众和风险选择：读者关心什么，就表达什么。

体系结构评价
Architecture Evaluation

评价什么

- 关键质量场景是否可满足
- 风险、敏感点和权衡点
- 依赖是否稳定、边界是否清晰
- 技术选择是否可替换
- 故障、扩展和部署是否可控

怎么评价

- 场景走查与质量属性工作坊
- ATAM：效用树、场景、风险
- 原型和基准测试
- 架构适应度函数
- 持续记录ADR与演进证据

评价的目标不是证明架构正确，而是尽早发现风险和权衡。

总结：体系结构是决策系统

Summary

明确组件与边界

表达连接与约束

保护关键质量属性

记录决策与权衡

保持边界可演进

用证据持续评价

好的架构不是“分层好看”，而是让关键质量可满足、变化被限制、决策可追溯。